Indian Institute of Technology Indore

Computer Science & Engineering

CS 216: Introduction To Blockchain

From Command Line to Programming: A Hands-on Approach

Prepared By (Team Core):

Kartik Budhani (240005022)

Krishnam Digga (240003043)

Rishan Gobse (240008023)

Arjun Dhamdhere (240005011)

March 10, 2026

Contents

1 Part 1: Legacy Address Transactions (P2PKH) 2
1.1 How the Payments Work . . . 2
1.2 Looking at the Code Rules (Scripts) . . . 2
1.2.1 The Lock (ScriptPubKey) . . . 2
1.2.2 The Key (ScriptSig) . . . 2
1.2.3 How Bitcoin Checks the Rules . . . 2
1.3 Debugger Screenshots . . . 3

2 Part 2: Newer SegWit Address Transactions 5
2.1 How the Payments Work . . . 5
2.2 Script Structure and Validation . . . 5
2.2.1 The Lock (ScriptPubKey) . . . 5
2.2.2 The Key (ScriptSig & Witness) . . . 5
2.2.3 How Bitcoin Checks the Rules . . . 5
2.3 Debugger Screenshots . . . 5

3 Part 3: Comparing the Results 7
3.1 Transaction Size Comparison . . . 7
3.2 Why is SegWit Better? . . . 7

4 Conclusion: Verification and Architectural Impact 8

Project Repository

GitHub Link:
`https://github.com/Kartik9112004/Assignment-2__Team-Core.git`

Note: The repository includes the main program source code, configuration files, and a `README.md` *containing run instructions and all team member roll numbers, as required by the assignment guidelines.*

1 Part 1: Legacy Address Transactions (P2PKH)

1.1 How the Payments Work

In this step, we created three old-style (legacy) Bitcoin addresses: A, B, and C. We gave some starting money to Address A. Then, we made a chain of payments. We took the leftover money from Address A and sent it to Address B.

- **Payment 1 (A → B):** The ID for this payment is `28733dada92b4e4c2cf95c6ef05c9c65db8ebc4ebaee2c12be98fbbee2de70d5`. Address B got the money in the first slot (`vout: 0`).
- **Payment 2 (B → C):** We used the money from `28733dada92b4e4c2cf95c6ef05c9c65db8ebc4ebaee2c12be98fbbee2de70d5` (at slot `vout: 0`) to send a payment to Address C. The ID for this final payment is `8d1207d244f1ee9f700040ba58e6cfeb08321534381e3860e9eba4bd3f65cb2f`.

1.2 Looking at the Code Rules (Scripts)

1.2.1 The Lock (ScriptPubKey)

This is the rule that locks the money for Address B. We got it from the first payment:

```
OP_DUP OP_HASH160 648e8d32ca2b105cf2164a9d17cffade6b01ce07
    OP_EQUALVERIFY OP_CHECKSIG
```

How it works: `OP_DUP OP_HASH160 <PubKeyHash> OP_EQUALVERIFY OP_CHECKSIG`
This acts like a puzzle. To spend the money, the owner must provide a Public Key that matches the hidden hash, and a real digital signature.

1.2.2 The Key (ScriptSig)

This is the "key" that Address B uses to unlock the money to send it to Address C:

```
30440220560
    a691c1fe870df740f8bd54dcc3235c898b9fc0213f229fbab87d9412842360220

731e293d8320e83e721a75fed573dd3facac45ac381f45bb4738c6edb0baf57a[
    ALL]
02c6dcdd06b2ded475b81073328ef0387904a5c5f40147e92bc046d9f1b37161b3
```

How it works: `<Signature> <PublicKey>`
This part simply gives Address B's digital signature and full Public Key to prove they own the money.

1.2.3 How Bitcoin Checks the Rules

To make sure the payment is honest, the Bitcoin network joins the Key and the Lock together and tests them step-by-step:

1. It puts the `<Signature>` and `<PublicKey>` on the table.
2. `OP_DUP` makes a copy of the Public Key.
3. `OP_HASH160` scrambles (hashes) the copied Public Key.

4. It brings out the expected hash from the lock script.
5. `OP_EQUALVERIFY` checks if the calculated hash and the expected hash are exactly the same.
6. `OP_CHECKSIG` checks if the digital signature is real. If everything is correct, the payment is approved.

1.3 Debugger Screenshots

```
(.venv) kartikbudhani@Kartiks-MacBook-Air TeamCore % btcdeb --tx=0200000001df80e1246aff6048f5622965ac74df20d8f9149bf54516b21de7db4bcca4605d000000006a473044022057895241953d3
4efaaf532cbbb3fe7c8f3e6f859014d221e9e73a15b7eaf2ebc022079e77748f23400a2ba5ab87f0ec850b7c5f5b3165ec77dc284b3625991166a3401210236 5d47980abfc28e4a03e8f24d785f466b018b01dc6967a
b660048e934093a62fdffffff01e07b9a3b000000001976a9141084fecbb8da6e66fd4f19e7bf43846fe030184388ac00000000 --txin=0200000001e70589119b249d8b1559 6da1d1a2a88b566d1efb9d8da307e2e
82809605bb5f700000006a473044022069633 2d004e4376f84cec57aca886fdf6b9b0d66b98d916c0e7815bf3d719b0c022009810 8e6dd92a17b3a2484c686a2494c769a7bb00176f9dbe6d99e48baaae4f00121026
edb820b42fa14544ee021e259f44910fec497b4acdc3e2e4952a235776344ecfdffffff01f0a29a3b000000001976a91491bafcfad6170d511e3b336561f5ad8eb845b2e588ac00000000
btcdeb 5.0.24 -- type `btcdeb -h` for start up options
LOG: signing segwit taproot
notice: btcdeb has gotten quieter; use --verbose if necessary (this message is temporary)
input tx index = 0; tx input vout = 0; value = 999990000
got witness stack of size 0
8 op script loaded. type `help` for usage information
script                                                              |  stack
--------------------------------------------------------------------+-------
304402205789524 1953d34efaaf532cbbb3fe7c8f3e6f859014d221e9e73a15... |
02365d47980abfc28e4a03e8f24d785f466b018b01dc6967ab660048e934093a62 |
<<< scriptPubKey >>>                                                |
OP_DUP                                                              |
OP_HASH160                                                          |
91bafcfad6170d511e3b336561f5ad8eb845b2e5                            |
OP_EQUALVERIFY                                                      |
OP_CHECKSIG                                                         |
#0000 304402205789524 1953d34efaaf532cbbb3fe7c8f3e6f859014d221e9e73a15b7eaf2ebc022079e77748f23400a2ba5ab87f0ec850b7c5f5b3165ec77dc284b3625991166a3401
btcdeb> step
                <> PUSH stack 304402205789524 1953d34efaaf532cbbb3fe7c8f3e6f859014d221e9e73a15b7eaf2ebc022079e77748f23400a2ba5ab87f0ec850b7c5f5b3165ec77dc284b3625991166a3401
script                                                              |                                                                 stack
--------------------------------------------------------------------+-----------------------------------------------------------------
02365d47980abfc28e4a03e8f24d785f466b018b01dc6967ab660048e934093a62 | 304402205789524 1953d34efaaf532cbbb3fe7c8f3e6f859014d221e9e73a15...
<<< scriptPubKey >>>                                                |
OP_DUP                                                              |
OP_HASH160                                                          |
91bafcfad6170d511e3b336561f5ad8eb845b2e5                            |
OP_EQUALVERIFY                                                      |
OP_CHECKSIG                                                         |
#0001 02365d47980abfc28e4a03e8f24d785f466b018b01dc6967ab660048e934093a62
```

Figure 1: btcdeb step-by-step execution - Part 1.

```
btcdeb> step
                <> PUSH stack 02365d47980abfc28e4a03e8f24d785f466b018b01dc6967ab660048e934093a62
script                                                              |                                                                 stack
--------------------------------------------------------------------+-----------------------------------------------------------------
<<< scriptPubKey >>>                                                | 02365d47980abfc28e4a03e8f24d785f466b018b01dc6967ab660048e934093a62
OP_DUP                                                              | 304402205789524 1953d34efaaf532cbbb3fe7c8f3e6f859014d221e9e73a15...
OP_HASH160                                                          |
91bafcfad6170d511e3b336561f5ad8eb845b2e5                            |
OP_EQUALVERIFY                                                      |
OP_CHECKSIG                                                         |
<<< scriptPubKey >>>
btcdeb> step
script                                                              |                                                                 stack
--------------------------------------------------------------------+-----------------------------------------------------------------
OP_DUP                                                              | 02365d47980abfc28e4a03e8f24d785f466b018b01dc6967ab660048e934093a62
OP_HASH160                                                          | 304402205789524 1953d34efaaf532cbbb3fe7c8f3e6f859014d221e9e73a15...
91bafcfad6170d511e3b336561f5ad8eb845b2e5                            |
OP_EQUALVERIFY                                                      |
OP_CHECKSIG                                                         |
#0003 OP_DUP
btcdeb> step
                <> PUSH stack 02365d47980abfc28e4a03e8f24d785f466b018b01dc6967ab660048e934093a62
script                                                              |                                                                 stack
--------------------------------------------------------------------+-----------------------------------------------------------------
OP_HASH160                                                          | 02365d47980abfc28e4a03e8f24d785f466b018b01dc6967ab660048e934093a62
91bafcfad6170d511e3b336561f5ad8eb845b2e5                            | 02365d47980abfc28e4a03e8f24d785f466b018b01dc6967ab660048e934093a62
OP_EQUALVERIFY                                                      | 304402205789524 1953d34efaaf532cbbb3fe7c8f3e6f859014d221e9e73a15...
OP_CHECKSIG                                                         |
#0004 OP_HASH160
btcdeb> step
                <> POP  stack
                <> PUSH stack 91bafcfad6170d511e3b336561f5ad8eb845b2e5
script                                                              |                                                                 stack
--------------------------------------------------------------------+-----------------------------------------------------------------
91bafcfad6170d511e3b336561f5ad8eb845b2e5                            |                                 91bafcfad6170d511e3b336561f5ad8eb845b2e5
OP_EQUALVERIFY                                                      | 02365d47980abfc28e4a03e8f24d785f466b018b01dc6967ab660048e934093a62
OP_CHECKSIG                                                         | 304402205789524 1953d34efaaf532cbbb3fe7c8f3e6f859014d221e9e73a15...
#0005 91bafcfad6170d511e3b336561f5ad8eb845b2e5
```

Figure 2: btcdeb step-by-step execution - Part 2.

```
btcdeb> step
                <> PUSH stack 91bafcfad6170d511e3b336561f5ad8eb845b2e5
script                                                          |                                                       stack
----------------------------------------------------------------+-------------------------------------------------------
OP_EQUALVERIFY                                                  |                          91bafcfad6170d511e3b336561f5ad8eb845b2e5
OP_CHECKSIG                                                     |                          91bafcfad6170d511e3b336561f5ad8eb845b2e5
                                                                | 02365d47980abfc28e4a03e8f24d785f466b018b01dc6967ab660048e934093a62
                                                                | 304402205789524195 3d34efaaf532cbbb3fe7c8f3e6f859014d221e9e73a15...
#0006 OP_EQUALVERIFY
btcdeb> step
                <> POP  stack
                <> POP  stack
                <> PUSH stack 01
                <> POP  stack
script                                                          |                                                       stack
----------------------------------------------------------------+-------------------------------------------------------
OP_CHECKSIG                                                     | 02365d47980abfc28e4a03e8f24d785f466b018b01dc6967ab660048e934093a62
                                                                | 304402205789524195 3d34efaaf532cbbb3fe7c8f3e6f859014d221e9e73a15...
#0007 OP_CHECKSIG
btcdeb> step
EvalChecksig() sigversion=0
Eval Checksig Pre-Tapscript
GenericTransactionSignatureChecker::CheckECDSASignature(71 len sig, 33 len pubkey, sigversion=0)
  sig         = 304402205789524195 3d34efaaf532cbbb3fe7c8f3e6f859014d221e9e73a15b7eaf2ebc022079e77748f23400a2ba5ab87f0ec850b7c5f5b3165ec77dc284b3625991166a3401
  pub key     = 02365d47980abfc28e4a03e8f24d785f466b018b01dc6967ab660048e934093a62
  script code = 76a91491bafcfad6170d511e3b336561f5ad8eb845b2e588ac
  hash type   = 01 (SIGHASH_ALL)
SignatureHash(nIn=0, nHashType=01, amount=999990000)
- sigversion = SIGVERSION_BASE (non-segwit style)
 << txTo.vin[nInput=0].prevout = COutPoint(5d60a4cc4b, 0)
(SerializeScriptCode)
 << scriptCode.size()=25 - nCodeSeparators=0
 << script:76a91491bafcfad6170d511e3b336561f5ad8eb845b2e588ac
 << txTo.vin[nInput].nSequence = 4294967293 [0xfffffffd]
  sighash     = 4e193f5df1dd24df2f5d25989e9d41098cb7fa2d5fe41300507d67f4f14c4e4d
  pubkey.VerifyECDSASignature(sig=304402205789524195 3d34efaaf532cbbb3fe7c8f3e6f859014d221e9e73a15b7eaf2ebc022079e77748f23400a2ba5ab87f0ec850b7c5f5b3165ec77dc284b3625991166a
34, sighash=4e193f5df1dd24df2f5d25989e9d41098cb7fa2d5fe41300507d67f4f14c4e4d):
  result: success
                <> POP  stack
                <> POP  stack
                <> PUSH stack 01
```

Figure 3: btcdeb step-by-step execution - Part 3.

```
script                                                          |                                                       stack
----------------------------------------------------------------+-------------------------------------------------------
                                                                |                                                          01
btcdeb> step
script                                                          |                                                       stack
----------------------------------------------------------------+-------------------------------------------------------
                                                                |                                                          01
btcdeb> step
at end of script
```

Figure 4: btcdeb step-by-step execution - Part 4.

2 Part 2: Newer SegWit Address Transactions

2.1 How the Payments Work

We created three newer addresses called SegWit addresses (A', B', and C'). We gave starting money to Address A'.

- **Payment 1 (A' → B'):** The ID for this payment is `2eb26b9d06453634186090c4c61d3819cf19e43502ad4767cd166e3d9037db32`.
- **Payment 2 (B' → C'):** We used the money from `2eb26b9d06453634186090c4c61d3819cf19e43502ad4767cd166e3d9037db32` at slot `vout: 0` to send money to Address C'. The final ID was `7ca3321f74e2c1ce319b47a33a7cced224281f43b9b39261d554d17f9db3cdab`.

2.2 Script Structure and Validation

2.2.1 The Lock (ScriptPubKey)

For these addresses, the lock is very simple:

```
OP_HASH160 4ea6b95bcd1075b197dba7517c97d5fb35e68bb0 OP_EQUAL
```

Structure: `OP_HASH160 <ScriptHash> OP_EQUAL`
The person sending the money doesn't even need to know this is a special SegWit payment. They just send it to a normal-looking address.

2.2.2 The Key (ScriptSig & Witness)

Older payments put the heavy signature data inside the main block. SegWit removes the heavy signature and puts it in a separate space called "Witness".

```
ScriptSig: 16001441b6fbe0cbb957d3d4153c889451f606fa6607d2

Witness Data:
1. 304402206
   f01391862f0784f070ba44abfca86337ba5728da576d9e97a23ded5743fa8830
   22068408
   e346f295194dee6925a17446275bc9f39b56b27153e2057bdac7ea7ef3a01
2. 026
   eb7909dbe98c9ed4d444b31df1613bcfcb807d58cc11526cac357a65138c567
```

Structure:
The main `ScriptSig` area only has a tiny reference code. The real unlocking keys (the Signature and the Public Key) are kept outside the main data in the `Witness` area.

2.2.3 How Bitcoin Checks the Rules

First, the system checks if the tiny reference code matches the lock. Once it passes, the system sees that this is a special SegWit payment. It then goes to the separate `Witness` area to check the real signature. It works just like the old way, but the heavy signature data doesn't take up expensive space in the main block.

2.3 Debugger Screenshots

```
(.venv) kartikbudhani@Kartiks-MacBook-Air TeamCore % btcdeb --tx=020000000001012104 59a906326912a143b6faffb20c2cb962ace500a84701be3cb0739e970a300000000017160014 3ae6435d5afc0
bc0471dca3fba79e50d14ea684bfdffffff01e07b9a3b0000000017a9146c59877af887097323681c0b5be23baf9858149a870247304402207d5708922e48fb87598dab4d402d77c066952dbbafc0a4adc63bee1ba32
6df7b02205cf2852478f4f9569656d337d01a59babeb9466bde0329cf448fe7db69124eef01210373d7c8d4b8529017dc9ec26aece899a2c91b354f7e81b5ca66e9fcd722f6a02e00000000 --txin=0200000000010
1aa9e81f0988d5859d2132ef721e15bd05439d159da534223425c2cdb2d836da200000000017160014 0b87531f19aa0f24643486006e188d68731e674bfdffffff01f0a29a3b0000000017a9147f5a5781cedf00f4c63
9994c25da757151d3737c870247304402202c95d2c5a155e5c659da1dd30dc6053b2a71178f3bcdfa7aadb4864069d923320220310cd18f66bda5fa64d2df33eca065360f8338043a9770d88b05fae03765eedb01210
3fb40df158a2822766357 6b3ab4483d400ee48f2558c5bc6564d6fc63f104928500000000
btcdeb 5.0.24 -- type `btcdeb -h` for start up options
LOG: signing segwit taproot
notice: btcdeb has gotten quieter; use --verbose if necessary (this message is temporary)
input tx index = 0; tx input vout = 0; value = 999990000
got witness stack of size 2
script sig non-empty; embedded P2SH (extracting payload)
hash source = 00143ae6435d5afc0bc0471dca3fba79e50d14ea684b
22 bytes (P2WPKH)
valid script
- generating prevout hash from 1 ins
[+] COutPoint(300a979e73, 0)
note: there is a for-clarity preamble (use --verbose for details)
5 op script loaded. type `help` for usage information
script                                   |                                                                  stack
-----------------------------------------+------------------------------------------------------------------
OP_DUP                                   | 0373d7c8d4b8529017dc9ec26aece899a2c91b354f7e81b5ca66e9fcd722f6a02e
OP_HASH160                               | 304402207d5708922e48fb87598dab4d402d77c066952dbbafc0a4adc63bee1...
3ae6435d5afc0bc0471dca3fba79e50d14ea684b |
OP_EQUALVERIFY                           |
OP_CHECKSIG                              |
#0000 OP_DUP
btcdeb> step
                <> PUSH stack 0373d7c8d4b8529017dc9ec26aece899a2c91b354f7e81b5ca66e9fcd722f6a02e
script                                   |                                                                  stack
-----------------------------------------+------------------------------------------------------------------
OP_HASH160                               | 0373d7c8d4b8529017dc9ec26aece899a2c91b354f7e81b5ca66e9fcd722f6a02e
3ae6435d5afc0bc0471dca3fba79e50d14ea684b | 0373d7c8d4b8529017dc9ec26aece899a2c91b354f7e81b5ca66e9fcd722f6a02e
OP_EQUALVERIFY                           | 304402207d5708922e48fb87598dab4d402d77c066952dbbafc0a4adc63bee1...
OP_CHECKSIG                              |
#0001 OP_HASH160
```

Figure 5: btcdeb step-by-step execution - Part 1.

```
btcdeb> step
                <> POP  stack
                <> PUSH stack 3ae6435d5afc0bc0471dca3fba79e50d14ea684b
script                                   |                                                                  stack
-----------------------------------------+------------------------------------------------------------------
3ae6435d5afc0bc0471dca3fba79e50d14ea684b |                                3ae6435d5afc0bc0471dca3fba79e50d14ea684b
OP_EQUALVERIFY                           | 0373d7c8d4b8529017dc9ec26aece899a2c91b354f7e81b5ca66e9fcd722f6a02e
OP_CHECKSIG                              | 304402207d5708922e48fb87598dab4d402d77c066952dbbafc0a4adc63bee1...
#0002 3ae6435d5afc0bc0471dca3fba79e50d14ea684b
btcdeb> step
                <> PUSH stack 3ae6435d5afc0bc0471dca3fba79e50d14ea684b
script                                   |                                                                  stack
-----------------------------------------+------------------------------------------------------------------
OP_EQUALVERIFY                           |                                3ae6435d5afc0bc0471dca3fba79e50d14ea684b
OP_CHECKSIG                              |                                3ae6435d5afc0bc0471dca3fba79e50d14ea684b
                                         | 0373d7c8d4b8529017dc9ec26aece899a2c91b354f7e81b5ca66e9fcd722f6a02e
                                         | 304402207d5708922e48fb87598dab4d402d77c066952dbbafc0a4adc63bee1...
#0003 OP_EQUALVERIFY
btcdeb> step
                <> POP  stack
                <> POP  stack
                <> PUSH stack 01
                <> POP  stack
script                                   |                                                                  stack
-----------------------------------------+------------------------------------------------------------------
OP_CHECKSIG                              | 0373d7c8d4b8529017dc9ec26aece899a2c91b354f7e81b5ca66e9fcd722f6a02e
                                         | 304402207d5708922e48fb87598dab4d402d77c066952dbbafc0a4adc63bee1...
#0004 OP_CHECKSIG
btcdeb> step
EvalChecksig() sigversion=1
Eval Checksig Pre-Tapscript
GenericTransactionSignatureChecker::CheckECDSASignature(71 len sig, 33 len pubkey, sigversion=1)
  sig         = 304402207d5708922e48fb87598dab4d402d77c066952dbbafc0a4adc63bee1ba326df7b02205cf2852478f4f9569656d337d01a59babeb9466bde0329cf448fe7db69124eef01
  pub key     = 0373d7c8d4b8529017dc9ec26aece899a2c91b354f7e81b5ca66e9fcd722f6a02e
  script code = 76a9143ae6435d5afc0bc0471dca3fba79e50d14ea684b88ac
  hash type   = 01 (SIGHASH_ALL)
SignatureHash(nIn=0, nHashType=01, amount=999990000)
- sigversion == SIGVERSION_WITNESS_V0
  sighash     = a2f19270977fba0c52c8c364d28916fd8ddee74f11b41dad974cd0c4d192ceb1
  pubkey.VerifyECDSASignature(sig=304402207d5708922e48fb87598dab4d402d77c066952dbbafc0a4adc63bee1ba326df7b02205cf2852478f4f9569656d337d01a59babeb9466bde0329cf448fe7db69124e
ef, sighash=a2f19270977fba0c52c8c364d28916fd8ddee74f11b41dad974cd0c4d192ceb1):
  result: success
```

Figure 6: btcdeb step-by-step execution - Part 2.

```
                <> POP  stack
                <> POP  stack
                <> PUSH stack 01
script                                   |                                                                  stack
-----------------------------------------+------------------------------------------------------------------
                                         |                                                                     01
btcdeb> step
script                                   |                                                                  stack
-----------------------------------------+------------------------------------------------------------------
                                         |                                                                     01
btcdeb> step
at end of script
```

Figure 7: btcdeb step-by-step execution - Part 3.

3 Part 3: Comparing the Results

3.1 Transaction Size Comparison

Using our Python code, we checked the file sizes of the old payment and the new SegWit payment:

Transaction Type	Raw Size (Bytes)	Virtual Size (vBytes)	Weight (WU)
Legacy (Old)	191	191	764
SegWit (New)	215	134	533

Table 1: Size comparison of the payments.

3.2 Why is SegWit Better?

Why is the Virtual Size smaller?
Our test shows that the SegWit payment takes up much less "virtual space" (134 vBytes) than the old payment (191 vBytes), even though the raw file size is a bit bigger.

This happens because the Segregated Witness (SegWit) upgrade restructures transaction data by completely separating the heavy signature (witness) data from the main transaction. In an old legacy payment, all data (including the large unlocking signature) is in the main area. Every byte here is heavy—it gets multiplied by 4 to calculate its "Weight".

In a SegWit payment, the large signature is moved to the separate Witness area. Data in the Witness area gets a 75% discount (it is not multiplied by 4), which drastically reduces the size counted against block limits.

$$\text{Weight} = (\text{Main Space Size} \times 4) + \text{Witness Space Size} \tag{1}$$

Because of this discount, the total Weight drops heavily (from 764 to 533). The network calculates the final Virtual Size (vSize) by dividing the Weight by 4:

$$\text{vSize} = \left\lceil \frac{\text{Weight}}{4} \right\rceil \tag{2}$$

This makes the final vSize much smaller.

What are the main benefits of this?

- **Cheaper Fees and More Space:** Miners charge fees based on Virtual Size, not raw size. Because the vSize is smaller, SegWit payments are much cheaper to send. Also, since they take up less main space, more payments can fit into a single Bitcoin block.
- **Fixing the Changing ID Bug:** In old payments, hackers could slightly alter the signature to change the payment's ID before it was confirmed. By moving the signature completely outside the main data, SegWit locks the ID so it cannot be changed.

- **Faster Networks:** Because the changing ID bug was fixed, developers could safely build fast networks on top of Bitcoin (like the Lightning Network) for instant, cheap, everyday payments.
- **Easier Upgrades:** SegWit introduced "version numbers" for scripts. This makes it much easier for developers to add new features to Bitcoin in the future without breaking the old rules.

4 Conclusion: Verification and Architectural Impact

Testing our P2SH-SegWit transaction with the `btcdeb` debugger clearly shows how the Segregated Witness upgrade works. In an older Legacy transaction, the signature and public key are stored right inside the main transaction script. SegWit, however, moves this data into a separate "witness" section.

By feeding our parent and child transaction IDs into the debugger (using `--tx` and `--txin`), we can watch it verify the transaction in two simple steps:

Phase 1: Checking the P2SH Wrapper

At first, the debugger doesn't even look at our signatures. It only pushes a **Redeem Script** to the stack to make sure its hash matches the destination address. Once this step passes (returns `TRUE`), the debugger sees the `00` SegWit marker and moves to the second phase.

Phase 2: Verifying the Witness Data

Now that the outer shell is unlocked, the debugger automatically pulls in our hidden **witness data** (our 70-byte Signature and 33-byte Public Key). It creates a fresh stack and runs a standard verification check. At the end of this step, the stack outputs a `1` (TRUE), proving the signature is valid and the coins can be spent.

Why This Matters (The Benefit)

Because SegWit separates the heavy signature data, it uses a new size measurement called virtual bytes (vbytes). This makes the transaction take up less space in a block, which directly lowers the transaction fees we have to pay. As a bonus, separating the signature also prevents bad actors from slightly altering (or "malleating") the transaction ID before it gets confirmed.